

Question 1

Part a.

This schedule is not conflict serializable, that is because there is a read-write conflict where $r_1(X)$ is read by T_1 before $w_2(X)$ is written by T_2 . There is also one more read-write conflict where $r_2(Y)$ is read by T_2 before $w_1(Y)$ is written by T_1 . There are also two write-write conflicts here, but both already mention the $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$ conflicts that we got from the read-write ones already. Note how we got a cycle $T_1 \rightarrow T_2 \rightarrow T_1$ (i.e. $T_1 \leftrightarrow T_2$) which indicates that the schedule is not conflict-serializable, i.e. there is no serial equivalent.

Part b.

This schedule is serializable on the base of semantic equivalence; Here is an example:

$r_1(X), eval_1(X := X - 10), w_1(X), r_1(Y), eval_1(Y := Y + 10), w_1(Y),$
 $r_2(Y), eval_2(Y := Y - 20), w_2(Y), r_2(X), eval_2(X := X - Y), w_2(X).$

This means that if we take evaluations that both transactions do into account, the result changes on the database are the same, i.e. equivalent.

Question 2

Part a.

The schedule is indeed recoverable. A schedule is not recoverable iff some transaction T_i wrote to some value X , then some other transaction T_j read that X , and finally T_j commits before T_i did. Since T_j depended on what T_i had written, if it were to commit before T_i did, it is possible that T_i aborted, and thus rolling back T_j would break the ACID's durability constraint, which is that all the committed changes are permanent (can be overwritten).

Part b.

However, the schedule is not cascadeless since there exists a transaction, namely T_2 , which reads from some other transaction, namely T_1 , before it committed. To fix that I propose the schedule: $r_2(X), w_1(X), w_1(X), r_1(Z), w_2(X), w_3(X), r_3(Y), r_1(Y), c_1, c_2, c_3$ in which the following was edited: $r_2(X)$ moved to the start, right before the first $w_1(X)$. Total changes: 1 move. Note that the result is obviously not equivalent to the schedule before, and that we got a dirty write here too...

Part c.

My “fixed” schedule is not strict since there is a transaction that reads/writes to a variable which was already written to by some other transaction without that other transaction committing yet. To make the original schedule strict (and thus cascadeless too), I propose the following schedule: $w_1(X), w_1(X), r_1(Z), r_1(Y), c_1, r_2(X), w_2(X), c_2, w_3(X), r_3(Y), c_3$. Here the following changes were made: $r_1(Y)$ was moved to right after $r_1(Z)$ and then the c_1 as the next one right after that, then moved the $r_2(X)$ to right before the $w_2(X)$ and c_2 to right after $w_2(X)$. Total changes: 4 moves.

Question 3**Part a.**

T_1 : *readlock*(X), *writelock*(Y), *read*(X), *write*(Y), *readunlock*(X), *writeunlock*(Y),
 T_2 : *writelock*(Z), *readlock*(X), *write*(Z), *read*(X), *writeunlock*(Z), *readunlock*(X), and
 T_3 : *readlock*(Y), *writelock*(Z), *read*(Y), *write*(Z), *readunlock*(Y), *writeunlock*(Z) are the modified transactions where I added the locks. First phase is the locking only part, then the second phase is the unlocking only part, simple as that. As you can see these are also shared/exclusive where we got read locks (shared) and write locks (exclusive). Also, no conversion was even needed here in the first place (I think).

Part b.

Yes, the modified transactions can end up in a deadlock, to show this here is an example schedule: $b_2, wl_2(Z), b_3, rl_3(Y), wl_3(Z), b_1, rl_1(X), wl_1(Y), rl_2(X)$. As you can see, T_3 waits until T_2 unlocks Z, T_1 waits until T_3 unlocks Y, and T_2 waits until T_1 unlocks X → Deadlock.

Part c.

S_a : $b_1, b_2, b_3, r_1(X), w_2(Z), w_1(Y), r_3(Y), r_2(X), w_3(Z)$ is an example schedule that follows the basic timestamp-based protocol. S_b : $b_1, b_2, b_3, r_1(X), w_2(Z), r_3(Y), w_1(Y)$ is an example schedule where rule 2 was broken because at the moment of doing $w_1(Y)$, $ReadTS(Y) > TS(T_1)$ which causes the system to roll-back T_1 . This rule 2 break is also known as a read-write conflict. S_c : $b_1, b_2, b_3, r_1(X), w_1(Y), r_3(Y), w_3(Z), w_2(Z), r_2(X)$ is an example schedule where the Thomas write rule is used ignoring the $w_3(Z), w_2(Z)$ rule 2 write-write conflict. That is because if we were to roll-back T_3 , the result would be the same as if we ignored it, that is the reason why the Thomas write rule works.

Question 4

With ARIES we first do the redo phase, and then the undo phase. We redo from the minimal *Lsn* found in the DIRTY BUFFER TABLE which is *Lsn* 2 which then reapplies the changes from that point onwards if they weren't already on the disk. So the redo order goes as follows: *Lsn* 2 : T_1 writes *B*, then *Lsn* 3 : T_1 writes *C*, and lastly *Lsn* 6 : T_2 writes *B*. Next, we undo with the following order: *Lsn* 3 : undo T_1 write *C*, then *Lsn* 2 : undo T_1 write *B*, and lastly *Lsn* 1 : undo T_1 write *A*.